

Distributed Databases and the CAP Theorem

Advanced Databases

M.Sc. Tran Quang Toan

Faculty of Data Science and Artificial Intelligence

Bases and coordinates

1. Part 1: Why Distributed?
2. Part 2: Database Replication
3. Part 3: Database Sharding
4. Part 4: The CAP Theorem
5. Part 5: Hands-on Lab

Learning Objectives

By the end of this session, you will be able to:

- Understand why modern AI systems cannot rely on a single server.
- Distinguish between **Vertical** and **Horizontal** scaling.
- Explain the mechanics of **Replication** and **Sharding**.
- Decode the ”Impossible Triangle”: **The CAP Theorem**.

Recap: Where are we?

PostgreSQL Single Node (Sessions 1-7)

- Deep-dive into Storage, Indexing (B-Tree, GIST).
- Advanced Queries (Window Functions, CTEs).
- Strong Consistency via **ACID** properties on a local disk.

The Reality Check:

- What if the SSD fails?
- What if 1,000,000 users query your AI model simultaneously?
- *A single machine is a liability.*

Part 1: Why Distributed?

1.1 Vertical Scaling (Scale-Up)

Definition: Adding more power (CPU, RAM, SSD) to an existing server.

- **Pros:**
  - No architectural changes needed.
  - Easy to manage (Single OS).
- **Cons:**
  - **Exponential Cost:** Doubling performance often quadruples the price.
  - **Hardware Ceiling:** You eventually hit the physical limit of the motherboard.
  - **Single Point of Failure (SPOF).**

1.2 Horizontal Scaling (Scale-Out)

Definition: Connecting multiple standard servers to work as one logical unit.

- **Cost-Effective:** Uses "commodity hardware" (standard, cheaper servers).
- **High Availability:** If one server dies, others take over.
- **Infinite Growth:** Just keep adding nodes.

AI Context: Distributed Training & Inference

Modern LLMs (like GPT-4) or large-scale Recommendation Systems (TikTok, Amazon) require data to be spread across thousands of GPUs and Databases.

1.3 Single Point of Failure (SPOF)

Why do Banking and Production-grade AI systems fear a "Single Node"?

1. **Hardware Failure:** Disk crash, power supply blowout.
2. **Software Issues:** OS kernel panic, memory leaks.
3. **Maintenance Downtime:** Need to restart for security patches? The whole system goes offline.

"Anything that can go wrong will go wrong." – Murphy's Law

$\Rightarrow$ **Distributed Databases eliminate SPOF.**

1.4 Vertical vs. Horizontal: Summary

Feature	Vertical Scaling	Horizontal Scaling
Cost	Expensive (High-end)	Cheap (Commodity)
Reliability	Low (SPOF)	High (Redundancy)
Limit	Physical Ceiling	Virtually Unlimited
Complexity	Low	High (Network overhead)
Maintenance	Requires Downtime	Zero-downtime possible

Transition: How do these servers talk and sync data? → ***Replication***

Part 2: Database Replication

2.1 What is Database Replication?

Replication is the process of sharing information so as to ensure consistency between redundant resources.

- It involves copying data from one database server (Master/Primary) to one or more servers (Slaves/Replicas).
- **Why?**
  - To improve reliability and fault tolerance.
  - To improve read performance.

Key Goal

Multiple copies of the same data, living on different physical machines.

2.2 Master-Slave (Primary-Replica) Model

The most common replication architecture:

- **Master (Primary):**
  - Handles all **Write** operations (INSERT, UPDATE, DELETE).
  - Acts as the "Source of Truth."
- **Slave (Replica):**
  - Handles **Read** operations (SELECT).
  - Synchronizes data from the Master.
  - Multiple slaves can be attached to one master.

2.3 Read-Scale Architecture

In AI applications (e.g., a Recommendation API), reads usually outnumber writes by 100:1.

- **Load Balancing:** Distribute SELECT queries across 5 replicas instead of 1 master.
- **Performance:** Master is freed up to process heavy write transactions.
- **AI Inference:** Use replicas to fetch feature vectors for model prediction.

2.4 Synchronous Replication

Mechanism: Master waits for confirmation from the Slave before committing the transaction.

- **Pros:**
  - No data loss (Zero-lag).
  - Strong consistency: All nodes have the same data at all times.
- **Cons:**
  - **Write Latency:** Transaction is only as fast as the slowest slave/network.
  - **Availability Risk:** If the slave is down, the master cannot complete writes.

2.5 Asynchronous Replication (Postgres Default)

Mechanism: Master commits the transaction locally and sends data to Slaves in the background.

- **Pros:**
  - Extremely fast writes (Master doesn't wait).
  - High availability: Master keeps working even if all Slaves die.
- **Cons:**
  - **Data Lag:** Slaves might be a few milliseconds/seconds behind.
  - **Data Loss Risk:** If Master crashes before data is sent, that data is lost.

2.6 Failover: What happens when Master dies?

Failover is the process of promoting a Slave to be the new Master.

1. **Detection:** Monitoring tools detect that the Master is unresponsive.
2. **Election:** A Slave (usually the one with the most recent data) is chosen.
3. **Promotion:** The Slave becomes the new Read/Write Master.
4. **Reconfiguration:** The application is pointed to the new Master IP.

Postgres Tools

Tools like **Patroni** or **Repmgr** automate this process.

2.7 Use Case: AI Training vs. Inference

- **Training Stage:**
  - Read massive datasets from **Read Replicas** to avoid locking the production database.
- **Inference Stage (Production):**
  - Global AI apps (like Uber or Netflix) replicate data to different geographical regions to reduce latency for local users.

Next: What if the database is too big for one disk? → ***Sharding***

Part 3: Database Sharding

3.1 What is Sharding?

Sharding is a method for distributing a single logical dataset across multiple databases.

- Unlike Replication (where every node has the *same* data), Sharding splits data so each node (**Shard**) holds a *unique subset*.
- It is the ultimate solution for **Horizontal Scaling**.

The "Large Table" Problem

If a table has 10 Billion rows, even with Indexes, B-Tree depth increases and performance degrades. Sharding breaks it into 10 tables of 1 Billion rows each.

3.2 Sharding vs. Horizontal Partitioning

Students often confuse these two. Let's clarify:

- **Horizontal Partitioning (Local):** Splitting a table into smaller chunks *within the same database instance*.
- **Sharding (Distributed):** Distributing those partitions across *different physical servers*.

Feature	Partitioning	Sharding
Location	Single Machine	Multiple Nodes
CPU/RAM Scaling	Limited to 1 Box	Scalable across many
Complexity	Moderate	High

3.3 The Sharding Key

The **Sharding Key** is the column that determines which shard a specific row belongs to.

- **Range-Based Sharding:**
  - e.g., customer_id 1-1000 $\rightarrow$ Shard A; 1001-2000 $\rightarrow$ Shard B.
  - *Risk:* Hotspots (e.g., all new users hit the same shard).
- **Hash-Based Sharding:**
  - e.g., $\text{hash}(\text{customer_id}) \ \% \ \text{N}$.
  - *Benefit:* Even distribution of data across all shards.

Critical Decision

Choosing the wrong Sharding Key is extremely difficult to fix later (requires "Resharding").

3.4 The Cost of Sharding

Sharding is not a ”silver bullet.” It introduces significant complexity:

1. **Cross-Shard Joins:** Joining tables located on different servers is extremely slow and should be avoided.
2. **Referential Integrity:** Foreign keys across shards are usually not supported by the DBMS.
3. **Transaction Complexity:** Distributed transactions (Two-Phase Commit) are required to maintain ACID across shards, which kills performance.

3.5 Sharding in the AI Lifecycle

In AI/ML Engineering, we shard when:

- **Feature Stores:** Storing billions of embeddings/vectors for real-time similarity search.
- **User Logs:** Tracking every click/action of millions of users to retrain models.
- **Example:** A social media app shards by $\mathrm{user_id}$ so that all data for one user stays on one shard, making "User Profile" queries fast.

Next: The theoretical limit of all these systems $\rightarrow$ ***The CAP Theorem***

Part 4: The CAP Theorem

4.1 Introduction to CAP Theorem

Proposed by Eric Brewer in 2000, the **CAP Theorem** states that a distributed data store can only provide two of the following three guarantees:

- **C (Consistency)**
- **A (Availability)**
- **P (Partition Tolerance)**

The Core Constraint

In the presence of a network partition (failure), a system must choose between **Consistency** and **Availability**. You cannot have both.

4.2 C: Consistency (Strong)

- **Definition:** Every read receives the most recent write or an error.
- **What it feels like:** The system behaves as if it were a single node. Once data is written, every subsequent query across any node returns that exact data.
- **Analogy:** A shared bank account. If I withdraw $50 in Hanoi, my wife in NYC should immediately see the balance decreased.

4.3 A: Availability

- **Definition:** Every request receives a (non-error) response, even if some nodes are down or out of sync.
- **What it feels like:** The system is always ”up” and responsive, though the data might be slightly ”stale” (outdated).
- **Analogy:** A social media feed. If a server in Europe is slow to sync, you might see a ”Like” from 1 minute ago instead of 1 second ago–but the app still loads.

4.4 P: Partition Tolerance

- **Definition:** The system continues to operate despite an arbitrary number of messages being dropped or delayed by the network between nodes.
- **The Reality:** In distributed systems, networks *will* fail (routers break, cables get cut).
- **Conclusion:** Since network partitions are inevitable in the real world, **P is mandatory**. We are forced to choose between **CP** or **AP**.

4.5 Making the Choice: CP vs. AP

CP (Consistency + Partition)

- If the network breaks, the system **shuts down** or returns errors to ensure data remains correct.
- *Use Case:* Banking, Stock Trading, Medical Records.

AP (Availability + Partition)

- If the network breaks, nodes stay online but provide **potentially inconsistent** data.
- *Use Case:* YouTube comments, Facebook Feed, Caching.

4.6 CAP Mapping of Popular Databases

System Type	Database Example	Priority
RDBMS	PostgreSQL (Single Node)	CA (Pure ACID)
CP System	MongoDB, HBase, Redis	Consistency
AP System	Cassandra, CouchDB	Availability

Note: Many modern DBs (like Postgres with Replication) can be tuned to be either more CP or more AP depending on configuration!

Part 5: Hands-on Lab

5.1 Lab Scenario: The dvdrental Cluster

Objective: Set up a Primary-Replica cluster using Docker.

- **Node 1 (Primary):** Port 5432. Handles Read/Write.
- **Node 2 (Replica):** Port 5433. Read-only.

Step 1: Spin up the Cluster

Run a pre-configured Docker Compose file that enables streaming replication.

Step 2: Restore dvdrental to Primary

```
pg_restore -d dvdrental dvdrental.tar
```

5.2 Behind the Scenes: Configuration

To make replication work, we modify two main files on the **Primary** node:

1. **postgresql.conf**:

```
wal_level = replica
max_wal_senders = 10
hot_standby = on

```

2. **pg_hba.conf** (Security):

```
# Allow replica user to connect from the network
host replication replicator 0.0.0.0/0 md5

```

Mechanism: The Primary streams its **Write-Ahead Logs (WAL)** to the Replica, which replays them in real-time.

5.3 Verification Task

Let's test if our AI-driven "Rental Stats" sync across nodes.

1. **At Primary:** Update a customer's rental status.

```
UPDATE customer SET active = 0 WHERE customer_id = 1;

```

2. **At Replica:** Query the same customer immediately.

```
-- This should show active = 0
SELECT active FROM customer WHERE customer_id = 1;

```

3. **Attempt Write at Replica:**

```
DELETE FROM rental WHERE rental_id = 10;
-- ERROR: cannot execute DELETE in a read-only transaction

```